IIM AHMEDABAD / CLAUDE CODE MEETUP

Building fast without fooling yourself

How I used Claude Code in a domain where mistakes matter.

THE HONEST STARTING POINT

The repo moved fast.
The domain had to be earned.

STARTING FACT

Speed was real. Trust was not automatic.

OPERATING RULE

Evidence first. Production confidence later.

VELOCITY

308

commits

LEARNING WINDOW

11

weeks

BOUNDARY

0

live trades before safety was credible

The process became more important than the prompt.

WHY THIS IS HARD

In some domains, a wrong answer looks convincing

WHAT FOOLS YOU

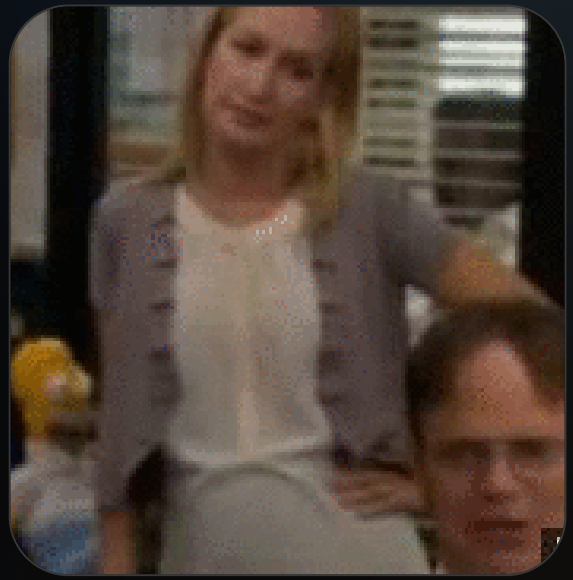
Looks good

A polished dashboard, profitable backtest, and confident explanation can still tell the wrong story.

WHAT CAN BE HIDING

Still wrong

Stale data, mismatched assumptions, or missing gates can turn confidence into risk.



The model sounded certain. Dwight asked for evidence.

The question is not "does it look right?" It is: can we prove it?

WHAT I BUILT

A decision system, not a dashboard

Every screen had to answer context, risk, or evidence.

1

Human screens

- Trading Day
- Discovery
- Backtest Lab
- Pipeline

2

Rules engine

Strategies, schedules, execution logic

3

Safety layer

Risk limits, approvals, fail-closed behavior

4

Evidence layer

Market state, backtests, decision history

5

Broker boundary

Actual order execution stays behind controls

AI helped build parts of this, but it does not get to bypass the checks.

MATURITY CURVE

The git history tells the story

01

Fast build

Vertical slices and broad feature commits.

02

Phases

Plans, trackers, smaller task boundaries.

03

Hardening

Audits, silent failure modes, fail-closed paths.

04

Realism

Backtests had to match production assumptions.

05

Governance

Approvals, lineage, and control-plane thinking.

The repo moved from "can we build it?" to "can we trust it?"

DOMAIN LEARNING

How I did not let the model fake finance

01 INTERROGATE

Ask from multiple angles

Use the model to surface assumptions, not to certify them.

02 WRITE

Turn uncertainty into artifacts

Docs, specs, and design notes became the working memory.

03 PACKAGE

Convert ideas into task packs

Acceptance criteria made work reviewable before code changed.

04 VERIFY

Treat first answers as drafts

Tests, audits, and corrections decided what was true.

WORKFLOW MATURITY

The workflow that actually worked

EARLY

Big prompt

1. **Ask broadly**
   The task sounded clear until the diff arrived.
2. **Big diff**
   Too much changed at once.
3. **Review painfully**
   Correctness was hard to see.

LATER

Smaller contract

1. **Spec + plan**
   Write the domain and acceptance criteria first.
2. **Isolate + parallelize**
   Use agents only when boundaries are clear.
3. **Test + review**
   Verify, then merge.

PRODUCTIVITY CAME FROM REDUCING CONFUSION, NOT GIVING THE MODEL UNLIMITED FREEDOM.

HARD LESSONS

The places I had to rethink everything

BACKTEST REALISM

Research had to match live behavior

If sizing assumptions drift, the backtest can tell a confident wrong story.

FAIL-CLOSED

Unclear should mean blocked

Permissive defaults are dangerous when money or decisions are involved.

GOVERNANCE

Approval needs a paper trail

Lineage, approvals, and decision history made the system explainable later.

The hardest work was truthfulness, not feature count.

WHAT YOU CAN REUSE

What you can use on Monday

Don't invent the domain

Use AI to expose gaps, not to hide them.

Write the risks

Name the things that must not go wrong.

Split the work

Turn broad ideas into task packs and acceptance checks.

Ask for proof

Tests, audits, and logs beat fluent explanations.

Keep humans in control

Risky actions need approval, not automation theater.

Use AI to learn faster, not to skip thinking.

Q & A

Questions?

Screens, code, workflow, mistakes, or what changed over time.